



PRoNTTo Manual

The *PRoNTTo* Development Group

John Ashburner
Carlton Chu
Andre Marquand
Janaina Mourao-Miranda
Christophe Phillips
Jonas Richiardi
Jane Rondina
Maria J. Rosa
Jessica Schrouff

Machine Learning & Neuroimaging Laboratory
Centre for Computational Statistics and Machine Learning
Computer Science department, UCL
Malet Place, London WC1E 6BT, UK
January 19, 2012

<http://www.mlnl.cs.ucl.ac.uk/pronto>

Contents

1	Introduction	7
1.1	Goals	7
1.2	Main contributors	8
1.3	Acknowledgements	9
I	Graphic User Interface	11
2	Data & design	13
2.1	Introduction	13
2.2	Defining groups, subjects, conditions, and modalities	13
2.3	Preprocessing	13
2.4	Results	13
3	Preprocessing & kernel calculation	15
4	Model & CV scheme specification	17
5	Mode & feature weights estimation	19
6	Results display	21
II	Batching system	23
7	Data & Design	25
7.1	Directory	25
7.2	Groups	25
7.2.1	Group	25
7.3	Masks	27
7.3.1	Modality	27
7.4	HRF overlap	27
7.5	HRF delay	27
7.6	Review	28
8	Feature set / Kernel	29
8.1	Load PRT.mat	29
8.2	Name	29
8.3	Modalities	29
8.3.1	Modality	29
9	Specify model	31
9.1	Load PRT.mat	31
9.2	Model name	31
9.3	Use kernels	31
9.4	Feature sets	31

9.5	Model Type	31
9.5.1	Classification	31
9.5.2	Regression	32
9.6	Cross-validation type	33
9.6.1	Leave one subject out	33
9.6.2	Leave one subject per group out	33
9.6.3	Leave one block out	33
9.6.4	Leave one run/session out	33
9.6.5	Custom	33
9.7	Data operations	34
9.7.1	Mean centre features	34
9.7.2	Other Operations	34
10	Run model	35
10.1	Load PRT.mat	35
10.2	Model name	35
III	Data processing examples	37
11	Data set 1	39
12	Data set 2	41
IV	Advanced topics	43
13	PRT structure	45
14	List of PRoNTo functions	49
14.1	prt.m	49
14.2	prt_apply_operation.m	49
14.3	prt_check_design.m	50
14.4	prt_compute_weights.m	50
14.5	prt_cv_model.m	50
14.6	prt_cv_opt_param.m	51
14.7	prt_data_conditions.m	51
14.8	prt_data_modality.m	52
14.9	prt_data_review.m	52
14.10	prt_defaults.m	53
14.11	prt_fs.m	53
14.12	prt_func2html.m	54
14.13	prt_get_defaults.m	54
14.14	prt_get_filename.m	54
14.15	prt_init_fs.m	54
14.16	prt_init_model.m	55
14.17	prt_latex.m	56
14.18	prt_load.m	56
14.19	prt_load_blocks.m	57
14.20	prt_model.m	57
14.21	prt_normalise_kernel.m	58
14.22	prt_permutation.m	58
14.23	prt_preproc.m	58
14.24	prt_remove_confounds.m	59
14.25	prt_stats.m	59
14.26	prt_struct2latex.m	59
14.27	prt_text_input.m	59
14.28	prt_ui_compute_weights.m	60

14.29prt_ui_cv_model.m	60
14.30prt_ui_design.m	61
14.31prt_ui_kernel_construction.m	61
14.32prt_ui_main.m	62
14.33prt_ui_model.m	62
14.34prt_ui_prepare_data.m	63
14.35prt_ui_prepare_datamod.m	63
14.36prt_ui_results.m	64
14.37prt_ui_reviewCV.m	64
14.38prt_ui_reviewmodel.m	65
14.39prt_ui_select_class.m	65
14.40prt_ui_select_reg.m	66
14.41machines	66
14.41.1 machines\prt_KRR.m	66
14.41.2 machines\prt_machine.m	66
14.41.3 machines\prt_machine_RT_bin.m	67
14.41.4 machines\prt_machine_gpml.m	67
14.41.5 machines\prt_machine_krr.m	68
14.41.6 machines\prt_machine_rvr.m	68
14.41.7 machines\prt_machine_svm_bin.m	69
14.41.8 machines\prt_rvr.m	69
14.41.9 machines\prt_weights.m	70
14.41.10machines\prt_weights_bin_linkkernel.m	71
14.41.11machines\prt_weights_svm_bin.m	71
14.42utils	71
14.42.1 utils\prt_centre_kernel.m	71
14.42.2 utils\prt_checkAlphaNumUnder.m	71
14.42.3 utils\prt_normalise_kernel.m	71

V Bibliography

Chapter 1

Introduction

This is where we introduce PRoNTo and explain what we wanted to do...
Cite a few important references like [1], or [2], or another [3] and obvious [4].

1.1 Goals

This comes from the Harvest application document.

Advances in neuroimaging techniques have radically changed the way neuroscientists address questions about functional anatomy, especially in relation to behavioural and clinical disorders. Many questions about brain function, previously investigated using electrophysiological recordings in animals can now be addressed non-invasively in humans. Such studies have yielded important results in cognitive neuroscience and neuropsychology. Amongst the various neuroimaging modalities available, Magnetic Resonance Imaging (MRI) has become widely used due to its relatively high spatial and temporal resolution, and because it is safe and non-invasive. By selecting specific MRI sequence parameters, different MR signals can be obtained from different tissue types, giving images with high contrast among organs, between normal and abnormal tissues and/or between activated and deactivated brain areas. MRI is often sub-categorized into structural MRI (sMRI) and functional MRI (fMRI). Structural and functional MRI data are inherently multivariate in nature, since each scan contains information about tissue structure, or brain activation, at thousands of measured locations (voxels). Considering that most brain functions are distributed processes involving a network of brain regions, it would seem desirable to use the spatially distributed information contained in the data to give a better understanding of brain functions in normal and abnormal conditions.

The typical analysis pipeline in neuroimaging is strongly rooted in a mass-univariate statistical approach, which assumes that activity in one brain region occurs independently from activity in other regions. Although this has yielded great insights over the years, and continues to be the tool of choice for data analysis, there is a growing recognition that the spatial dependencies among signal from different brain regions should be properly modeled. The effect of interest can be subtle and spatially distributed over the brain - a case of high-dimensional, multivariate data modeling for which conventional tools may lack sensitivity.

Therefore, there has been an increasing interest in investigating this spatially distributed information using multivariate pattern recognition methods. Where pattern recognition has been used in neuroimaging, it has led to fundamental advances in the understanding of how the brain represents information and has been applied to many diagnostic applications. For the latter, this approach can be used to predict the status of the patient scanned (healthy vs. diseased or disease A vs. B) and can provide the discriminating pattern leading to this classification. Several active areas of research in machine learning are crucially important for the difficult problem of neuroimaging data analysis: modelling of high-dimensional multivariate time series, sparsity, regularisation, dimensionality reduction, causal modeling, and ensembling to name a few. However, the application of pattern recognition approaches to the analysis of neuroimaging data is limited mainly by the lack of user-friendly and comprehensive tools available to the fundamental, cognitive, and clinical neuroscience communities. Furthermore, it is not uncommon for these

methods to be used incorrectly, with the most typical case being improper separation of training and testing datasets.

The aim of the project is to develop a toolbox based on machine learning techniques for the analysis of neuroimaging data (PRoNTo: Pattern Recognition Neuroimaging Toolbox). The toolbox will be a MATLABbased software package specifically designed for the analysis of brain imaging data including steps for feature extraction according to the experimental design, feature selection, machine training and testing. Initially it will focus on supervised learning approaches such as Support Vector Machines, Gaussian Processes, etc. Other techniques will be added once a consistent framework and data flow has been implemented.

The toolbox will facilitate the interaction between machine learning and neuroimaging communities. On one hand the machine learning community will be able to contribute to the toolbox with novel published machine learning algorithms. On the other hand the toolbox will provide a variety of tools for the neuroscience and clinical neuroscience communities, enabling them to ask new questions that cannot be easily investigated using existing statistical analysis tools. The toolbox code will be distributed for free, but as copyright software under the terms of the GNU General Public License as published by the Free Software Foundation.

1.2 Main contributors

PRoNTo is developed by the Machine Learning & Neuroimaging Laboratory, Computer Science department, University College London, UK (<http://www.mlnl.cs.ucl.ac.uk>) and associated researchers.

The main contributors, in alphabetical order, are:

Dr. John Ashburner is a reader at the Wellcome Trust Centre for Neuroimaging at the UCL Institute of Neurology. He is mainly interested in modeling brain anatomy from MR scans, and more recently in applying pattern recognition methods to make predictions about individual subjects. He is a co-developer of the SPM software (intra- and inter-subject registration, tissue classification, visualization and image file formats), which is used internationally by thousands of neuroimaging researchers. He has authored or co-authored 90 papers in international journals (h-index of 50) and written a number of book chapters;

Dr. Carlton Chu is a research fellow in brain imaging at the National Institute of Mental Health (NIMH), NIH. He received the B.Eng. degree (1st class Honours) from Auckland University, New Zealand, in 2002 and the master of Biomedical Engineering from University of New South Wales, Australia, in 2004. Carlton obtained a PhD in Neuroimaging method from University College London in 2009, working in the statistical methods group at the prestigious Wellcome Trust Centre for Neuroimaging, creators of the famous SPM program. There he developed innovative new pattern recognition methods to automatically detect the early stages of neurodegenerative diseases such as Alzheimers and Huntingdons just from structural brain images. In 2007, Carlton won the first prize in the 2nd Pittsburgh Brain Activity Interpretation Competition (PBAIC), a prestigious international competition involving the application of machine learning to the problem of classification of brain activity. He led a small research team to victory, acclaim from peers in the field, and the \$10K first prize. His current research interests include brain state decoding, neurodegenerative disease classification, and applying pattern recognition method to study brain networks;

Dr. Andre Marquand is a Post-Doctoral Research Fellow at the Centre for Neuroimaging Sciences, Kings College London and an Honorary Post-Doctoral Research Fellow at the Centre for Computational Statistics and Machine Learning at University College London. His research focuses on the application of probabilistic machine learning techniques to neuroimaging data, particularly for clinical applications. His recent work includes the application of multi-class and multi-modality pattern classification methods to neuroimaging and in particular to detecting the effects of psychotropic medication on patterns of brain activity;

Dr. Janaina Mourao-Miranda is a Wellcome Trust Senior Research Fellow at Centre for Computational Statistics and Machine Learning (CSML), UCL and at the Centre for Neu-

roimaging Sciences (CNS), KCL. Her research focuses on developing and applying pattern recognition methods to analyze neuroimaging data, in particular brain activation and structural patterns that distinguish between controls and patients. Recent work includes the development and application of spatio-temporal SVM, one-class SVM to detect patients as outliers and in-depth studies of kernel methods for brain decoding;

Dr. Christophe Phillips is FRS-FNRS Research Associate at the Cyclotron Research Centre and adjunct Assistant Professor at the Department of Electrical Engineering and Computer Science, University of Lige, Belgium. His research focuses on the processing of multi-modal neuroimaging data. Recent work within the field of brain decoding aimed at distinguishing between levels of consciousness in unresponsive patients or between typical and atypical Parkinson Disease patients using Positron Emission Tomography (PET) imaging, as well as tracking mnesic traces in trained healthy subjects with fMRI;

Dr. Jonas Richiardi is a post-doctoral research fellow, jointly affiliated to the EPFL engineering school (Medical Image Processing Laboratory) and the Geneva university hospital (Department of Radiology and Medical Informatics). His research interests include brain connectivity and resting-state networks analysis, interpretability of brain decoding results, functional biomarkers, learning with graphs, and machine learning for neuroimaging. He was co-chair of the 2010 Brain Decoding Workshop at the Int. Conf. on Pattern Recognition and a program chair of the Pattern Recognition in NeuroImaging workshop 2011;

Dr. Jane Rondina is a Wellcome Trust Post Doctoral Research Associate at Centre for Neuroimaging Sciences (CNS), KCL and researcher as an honorary member at Centre for Computational Statistics and Machine Learning (CSML), UCL. Her current work includes the development of a feature selection method for classification in neuroimaging and analysis of features stability. Her research interests also include the development of pattern recognition methods using data from different modalities / measures in neuroimaging;

Ms Maria J. Rosa is a PhD student at the Wellcome Trust Centre for Neuroimaging, Institute of Neurology, University College London, UK. Her areas of interest include Bayesian model selection methods for fMRI and Dynamic Causal Modelling, EEG-fMRI fusion and biophysical models of neurovascular coupling;

Ms Jessica Schrouff has a Master in Biomedical Engineering and is currently pursuing a PhD in neuroimaging at the Cyclotron Research Centre, University of Lige, Belgium, under the supervision of Dr C. Phillips. Her project focuses on the tracking of mnesic traces of learned images in trained healthy subjects with fMRI and EEG data, as well as the classification of patients from PET images.

1.3 Acknowledgements

PRoNTo v1 (2011) is the deliverable of a Pascal Harvest project coordinated by Dr. J. Mourao-Miranda and its development was possible with the financial and logistic support of

- the Department of Computer Science, University College London (<http://www.cs.ucl.ac.uk>);
- the Wellcome Trust (<http://www.wellcome.ac.uk/>);
- PASCAL2 (<http://www.pascal-network.org>) and its HARVEST programme;
- the Fonds de la Recherche Scientifique-FNRS (<http://www.fnrs.be>), Belgium;
- Swiss National Science Foundation (PP00P2-123438) and Center for Biomedical Imaging (CIBM) of the EPFL and Universities and Hospitals of Lausanne and Geneva.

PRoNTo is written for MATLAB X.Y (R20ZZb) and onwards. Some routine may need to be compiled for your specific OS.

Part I

Graphic User Interface

Chapter 2

Data & design

This is where we explain how data and design are defined within the GUI. **Lots of text to come!**

2.1 Introduction

blablabla

2.2 Defining groups, subjects, conditions, and modalities

As a general rule, group, subject, condition, and modality names should contain only alphanumeric characters (A-Z,a-z,0-9) or underscores _.

2.3 Preprocessing

more stuff

2.4 Results

Txt and equation $A = 10$.

Chapter 3

Preprocessing & kernel calculation

This is where we explain how data are preprocessed and a kernel built.
Lots of text to come!

Chapter 4

Model & CV scheme specification

This is where we explain how a model is specified, as well as the cross-validation (CV) scheme necessary to validate the model.

Lots of text to come!

Chapter 5

Mode & feature weights estimation

This is where we explain how we run the model and calculate the weights.
Lots of text to come!

Chapter 6

Results display

This is where we explain how you can display results within the GUI. In particular, what are all the nice results we can show/use to write good papers. Lots of text to come!

Part II

Batching system

Chapter 7

Data & Design

Specify the group(s) of data set.

7.1 Directory

Select a directory where the PRT.mat file containing the specified design and data matrix will be written.

7.2 Groups

Add data and design for one group. Click 'new' or 'repeat' to add another group.

7.2.1 Group

Specify data and design for the group.

Name

Name of the group. Example: 'Controls'.

Select by

Depending on the type of data at hand, you may have many images (scans) per subject, such as a fMRI time series, or you may have many subjects with only one or a small number of images (scans) per subject, such as PET images. If you have many scans per subject select the option 'subjects'. If you have one scan for many subjects select the option 'scans'.

Subjects Add subjects.

Subject Add new modality for this subject.

Modality Add new data modality.

NAME Name of modality. Example: 'BOLD'.

INTERSCAN INTERVAL Specify interscan interval (TR). The units should be seconds.

SCANS Select scans (images) for this modality. They must all have the same image dimensions, orientation, voxel size etc.

DATA & DESIGN Specify data and design.

Load SPM.mat Load design from SPM.mat (if you have previously specified the experimental design with SPM).

Specify design Specify design: scans (data), onsets and durations.

Units for design The onsets of events or blocks can be specified in either scans or seconds.

Conditions Specify conditions. You are allowed to combine both event- and epoch-related responses in the same model and/or regressor. Any number of condition (event or epoch) types can be specified. Epoch and event-related responses are modeled in exactly the same way by specifying their onsets [in terms of onset times] and their durations. Events are specified with a duration of 0. If you enter a single number for the durations it will be assumed that all trials conform to this duration. For factorial designs, one can later associate these experimental conditions with the appropriate levels of experimental factors.

Condition Specify condition: name, onsets and duration.

Name Name of condition.

Onsets Specify a vector of onset times for this condition type.

Durations Specify the event durations. Epoch and event-related responses are modeled in exactly the same way but by specifying their different durations. Events are specified with a duration of 0. If you enter a single number for the durations it will be assumed that all trials conform to this duration. If you have multiple different durations, then the number must match the number of onset times.

Multiple conditions Select the *.mat file containing details of your multiple experimental conditions.

If you have multiple conditions then entering the details a condition at a time is very inefficient. This option can be used to load all the required information in one go. You will first need to create a *.mat file containing the relevant information.

This *.mat file must include the following cell arrays (each 1 x n): names, onsets and durations. eg. names=cell(1,5), onsets=cell(1,5), durations=cell(1,5), then names2='SSent-DSpeak', onsets2=[3 5 19 222], durations2=[0 0 0 0], contain the required details of the second condition. These cell arrays may be made available by your stimulus delivery program, eg. COGENT. The duration vectors can contain a single entry if the durations are identical for all events.

Time and Parametric effects can also be included. For time modulation include a cell array (1 x n) called tmod. It should have a single number in each cell. Unused cells may contain either a 0 or be left empty. The number specifies the order of time modulation from 0 = No Time Modulation to 6 = 6th Order Time Modulation. eg. tmod3 = 1, modulates the 3rd condition by a linear time effect.

For parametric modulation include a structure array, which is up to 1 x n in size, called pmod. n must be less than or equal to the number of cells in the names/onsets/durations cell arrays. The structure array pmod must have the fields: name, param and poly. Each of these fields is in turn a cell array to allow the inclusion of one or more parametric effects per column of the design. The field name must be a cell array containing strings. The field param is a cell array containing a vector of parameters. Remember each parameter must be the same length as its corresponding onsets vector. The field poly is a cell array (for consistency) with each cell containing a single number specifying the order of the polynomial expansion from 1 to 6.

Note that each condition is assigned its corresponding entry in the structure array (condition 1 parametric modulators are in pmod(1), condition 2 parametric modulators are in pmod(2), etc. Within a condition multiple parametric modulators are accessed via each fields cell arrays. So for condition 1, parametric modulator 1 would be defined in pmod(1).name1, pmod(1).param1, and pmod(1).poly1. A second parametric modulator for condition 1 would be defined as pmod(1).name2, pmod(1).param2 and pmod(1).poly2. If there was also a parametric modulator for condition 2, then remember the first modulator for that condition is in cell array 1: pmod(2).name1, pmod(2).param1, and pmod(2).poly1. If some, but not all conditions are parametrically modulated, then the non-modulated indices in the pmod structure can be left blank. For example, if conditions 1 and 3 but not condition 2 are modulated, then specify pmod(1) and pmod(3). Similarly, if conditions 1 and 2 are modulated but there are 3 conditions overall, it is only necessary for pmod to be a 1 x 2 structure array.

EXAMPLE:

Make an empty pmod structure:

```
pmod = struct('name','','param','','poly',);
```

Specify one parametric regressor for the first condition:

```
pmod(1).name1 = 'regressor1';
```

```
pmod(1).param1 = [1 2 4 5 6];
```

```

pmod(1).poly1 = 1;
Specify 2 parametric regressors for the second condition:
pmod(2).name1 = 'regressor2-1';
pmod(2).param1 = [1 3 5 7];
pmod(2).poly1 = 1;
pmod(2).name2 = 'regressor2-2';
pmod(2).param2 = [2 4 6 8 10];
pmod(2).poly2 = 1;

```

The parametric modulator should be mean corrected if appropriate. Unused structure entries should have all fields left empty.

Covariates Select a .mat file containing your covariates (i.e. any other data/information you would like to include in your design). This file should contain a variable 'R' with the covariates.

No design Do not specify design. This option can be used for modalities (e.g. structural scans) that do not have an experimental design.

Scans Select this option if you have many subjects to spatially normalise, but there is one or a small number of scans for each subject.

Modality Specify modality, such as name and data.

Name Name of modality. Example: 'BOLD'.

Files Select scans (images) for this modality. They must all have the same image dimensions, orientation, voxel size etc.

Regression targets (per scans) Enter one regression target per scans. or enter the name of a variable This variable should be a vector [Nscans x 1], where Nsubjs is the number of subjects.

Covariates Select a .mat file containing your covariates (i.e. any other data/information you would like to include in your design). This file should contain a variable 'R' with the covariates.

7.3 Masks

Select first-level (pre-processing) mask for each modality. The name of the modalities should be the same as the ones entered for subjects/scans.

7.3.1 Modality

Specify name of modality and file for each mask.

Name

Name of modality. Example: 'BOLD'.

File

Select one mask for each modality.

7.4 HRF overlap

If using fMRI data please specify the width of the hemodynamic response function (HRF). This will be used to calculate the overlap between events. Leave as 0 for other modalities.

7.5 HRF delay

If using fMRI data please specify the delay of the hemodynamic response function (HRF). This will be used to calculate the overlap between events. Leave as 0 for other modalities.

7.6 Review

Would you like to review the design?.

Chapter 8

Feature set / Kernel

Compute feature set according to the design specified

8.1 Load PRT.mat

Select data/design structure file (PRT.mat).

8.2 Name

Target name for kernel matrix. This should contain only alphanumerical characters or underscores (-).

8.3 Modalities

Add modalities

8.3.1 Modality

Specify modality, such as name and data.

Name

Name of modality. Example: 'BOLD'. Must match design specification

Scans / Conditions

Which task conditions do you want to include in the kernel matrix? Select conditions: select specific conditions from the timeseries. All conditions: include all conditions extracted from the timeseries. All scans: include all scans for each subject. This may be used for modalities with only one scan per subject (e.g. PET), if you want to include all scans from an fMRI timeseries (assumes you have not already detrended the timeseries and extracted task components)

All scans No design specified. This option can be used for modalities (e.g. structural scans) that do not have an experimental design or for an fMRI design where you want to include all scans in the timeseries

All Conditions Include all conditions in this kernel matrix

Voxels to include

Specify which voxels from the current modality you would like to include

All voxels Use all voxels in the design mask for this modality

Specify mask file Select a mask for the selected modality.

Detrend

Type of temporal detrending to apply

None Do not detrend the data

Polynomial detrend Perform a voxel-wise polynomial detrend on the data (1 is linear detrend)

Order Enter the order for polynomial detrend (1 is linear detrend)

Discrete cosine transform Use a discrete cosine basis set to detrend the data.

Cutoff of high-pass filter (second) The default high-pass filter cutoff is 128 seconds (same as SPM)

Scale input scans

Do you want to scale the input scans to have a fixed mean (i.e. grand mean scaling)?

No scaling Do not scale the input scans

Specify from *.mat Specify a mat file containing the scaling parameters for each modality.

Chapter 9

Specify model

Construct model according to design specified

9.1 Load PRT.mat

Select data/design structure file (PRT.mat).

9.2 Model name

Name for model

9.3 Use kernels

Are the data for this model in the form of kernels/basis functions? If 'No' is selected, it is assumed the data are in the form of feature matrices

9.4 Feature sets

Enter the name of a feature set to include in this model. This can be kernel or a feature matrix.

9.5 Model Type

Select which kind of predictive model is to be used.

9.5.1 Classification

Specify classes and machine for classification.

Classes

Specify which elements belong to this class. Click 'new' or 'repeat' to add another class.

Class Specify which groups, modalities, subjects and conditions should be included in this class

Name Name for this class, e.g. 'controls'

Groups Add one group to this class. Click 'new' or 'repeat' to add another group.

Group Specify data and design for the group.

GROUP NAME Name of the group to include. Must exist in PRT.mat

SUBJECTS Subject numbers to be included in this class. Note that individual numbers (e.g. 1), or a range of numbers (e.g. 3:5) can be entered

CONDITIONS / SCANS Which task conditions do you want to include? Select conditions: select specific conditions from the timeseries. All conditions: include all conditions extracted from the timeseries. All scans: include all scans for each subject. This may be used for modalities with only one scan per subject (e.g. PET), if you want to include all scans from an fMRI timeseries (assumes you have not already detrended the timeseries and extracted task components)

Specify Conditions Specify the name of conditions to be included

Condition Specify condition:.

Name Name of condition to include.

All Conditions Include all conditions in this model

All scans No design specified. This option can be used for modalities (e.g. structural scans) that do not have an experimental design or for an fMRI design where you want to include all scans in the timeseries

Machine

Choose a prediction machine for this model

SVM Classification Binary support vector machine.

Arguments Arguments for `prt_machine_svm_bin`. You should use `-t 4` if you selected 'use kernels' option, and `-t 0` otherwise. See libSVM documentation for details.

Gaussian Process Classification Gaussian Process Classification

Arguments Arguments for `prt_machine_gpml`.

Random Forest Random Forest. Breiman, Leo (2001). "Random Forests".

Machine Learning 45:5-32. This is a wrapper around Peter Geurt's implementation in his Regression Tree package.

Ntrees Number of trees in the forest.

Custom machine Choose another prediction machine

Function Choose a function that will perform prediction.

Arguments Arguments for prediction machine.

9.5.2 Regression

Add group data and machine for regression.

Groups

Add one group to this regression model. Click 'new' or 'repeat' to add another group.

Group Specify data and design for the group.

Group name Name of the group to include. Must exist in PRT.mat

Subjects Subject numbers to be included in this class. Note that individual numbers (e.g. 1), or a range of numbers (e.g. 3:5) can be entered

Modality name Name of modality. We only allow one modality for regression model per group at this moment

Example: 'BOLD'. Must match design specification

Machine

Choose a prediction machine for this model

Kernel Ridge Regression Kernel Ridge Regression.

Regularization Regularization for `prr_machine_krr`.

Relevance Vector Regression Relevance Vector Regression. Tipping, Michael E.; Smola, Alex (2001).

"Sparse Bayesian Learning and the Relevance Vector Machine". Journal of Machine Learning Research 1: 211?244.

Custom machine Choose another prediction machine

Function Choose a function that will perform prediction.

Arguments Arguments for prediction machine.

9.6 Cross-validation type

Choose the type of cross-validation to be used

9.6.1 Leave one subject out

Leave a single subject out each cross-validation iteration

9.6.2 Leave one subject per group out

Leave out a single subject from each group at a time. Appropriate for repeated measures or paired samples designs.

9.6.3 Leave one block out

Leave out a single block or event from each subject each iteration. Appropriate for single subject designs.

9.6.4 Leave one run/session out

Leave out a single run (modality) from each subject each iteration. Appropriate for single subject designs with multiple runs/sessions.

9.6.5 Custom

Load a cross-validation matrix. Note that an interface will be provided for this functionality in a later release

9.7 Data operations

Specify operations to apply

9.7.1 Mean centre features

Select an operation to apply.

9.7.2 Other Operations

Include other operations?

No operations

No design specified. This option can be used for modalities (e.g. structural scans) that do not have an experimental design or for an fMRI design where you want to include all scans in the timeseries

Select Operations

Add zero or more operations to be applied to the data before the prediction machine is called. These are executed within the cross-validation loop (i.e. they respect training/test independence) and will be executed in the order specified.

Operation Select an operation to apply.

Chapter 10

Run model

Trains and tests the predictive machine using the cross-validation structure specified by the model.

10.1 Load PRT.mat

Select PRT.mat (file containing data/design structure).

10.2 Model name

Name of a model. Must match your entry in the
'Specify model' batch module.

Part III

Data processing examples

Chapter 11

Data set 1

This is where we explain how how to process data set 1.

Lots of text to come!

Chapter 12

Data set 2

This is where we explain how how to process data set 2.

Lots of text to come!

Part IV

Advanced topics

Chapter 13

PRT structure

This is how the main PRT structure is organised.
PRT

- group
 - gr_name
 - subject
 - subj_name()
 - modality()
 - mod_name
 - TR
 - scans
 - design
 - conds
 - cond_name()
 - onsets()
 - durations()
 - rt_trial()
 - scans()
 - blocks()
 - discardedscans()
 - hrfdiscardedscans()
 - stats
 - overlap
 - goodscans
 - discscans
 - meanovl
 - stdovl
 - mgoodovl
 - sgoodovl
 - goodovl
 - TR
 - unit
 - covar
- masks
 - mod_name

- fname
- fs
 - fs_name
 - k_file
 - id_col_names
 - fas
 - im
 - ifa
 - modality
 - mod_name
 - detrend
 - param_dt
 - mode
 - idfeat_fas
 - normalise
 - type
 - scaling
 - id_mat
- fas
 - mod_name
 - dat
 - detrend
 - param_dt
 - hdr
 - fname
 - dim
 - mat
 - pinfo
 - dt
 - n
 - descrip
 - private
 - idfeat_img
- model
 - model_name()
 - input()
 - use_kernel
 - type
 - machine
 - function
 - args
 - class
 - class_name()
 - group()
 - gr_name

- subj
 - num()
 - modality()
- fs
 - fs_name
- samp_idx
- targets
- targ_allscans
- cv_mat
- operations
- cv_type
- output()
 - fold
 - targets()
 - predictions()
 - stats()
 - con_mat
 - acc
 - c_acc
 - b_acc
 - acc_lb
 - acc_ub
 - func_val()
 - type()
 - alpha()
 - b()
 - totalSV()
- stats
 - con_mat
 - acc
 - c_acc
 - b_acc
 - acc_lb
 - acc_ub

Chapter 14

List of PRoNTo functions

This is the list of PRoNTo functions, including the subdirectories: `machines` and `utils`.

14.1 `prt.m`

Pattern Recognition for Neuroimaging Toolbox, PRoNTo.

This function initializes things for PRoNTo and provides some low level functionalities

14.2 `prt_apply_operation.m`

function to apply a data operation to the training, test and

```
in.train:      training data
in.tr_id:      id matrix for training data
in.use_kernel: are the data in kernelised form
in.tr_targets: training targets (optional field)
in.pred_type:  'classification' or 'regression' (required for tr_targets)
```

A test set may also be specified, which require the following fields:

```
in.test:      test data
in.testcov:   test covariance (only if use_kernel = true)
in.te_targets: test targets
in.te_id:     id matrix for test data
```

`opid` specifies the operation to apply, where:

- 1 = Temporal Compression
- 2 = Sample averaging (average samples for each subject/condition)
- 3 = Mean centre features over subjects
- 4 = Divide data vectors by their norm
- 5 = Perform a GLM (fMRI only)

N.B: - all operations are applied independently to training and test partitions
- see Chu et. al (2011) for mathematical descriptions of operations

1 and 2 and Shawe-Taylor and Cristianini (2004) for a description of operation 3.

References:

Chu, C et al. (2011) Utilizing temporal information in fMRI decoding: classifier using kernel regression methods. *Neuroimage*. 58(2):560-71.
 Shawe-Taylor, J. and Cristianini, N. (2004). *Kernel methods for Pattern analysis*. Cambridge University Press.

14.3 prt_check_design.m

FORMAT [conds] = prt_check_design(cond,tr,units,hrfoverlap)

Check the design and discards scans which are either overlapping between conditions or which do not respect a minimum time interval between conditions (due to the width of the HRF function).

INPUT

- cond : structure containing the names, durations and onsets of the conditions
- tr : interscan interval (TR)
- units : 1 for seconds, 0 for scans
- hrfoverlap : value to correct for BOLD overlap (in seconds)
- hrfdelay : value to correct for BOLD delay (in seconds)

OUTPUT

the same cond structure containing supplementary fields:

- scans : scans retained for further classification
- discardedscans: scans discarded because they overlapped between conditions
- hrfdiscardedscans: scans discarded because they didn't respect the minimum time interval between conditions
- blocks: represents the grouping of the stimuli (for cross-validation)
- stats: struct containing the original time intervals, the time interval with only the 'good' scans, their means and standard deviation

14.4 prt_compute_weights.m

FORMAT prt_compute_weights(PRT,in)

This function calls prt_weights to compute weights

14.5 prt_cv_model.m

Function to run a cross-validation structure on a given model

Inputs:

PRT containing the specified model plus the following arguments:

in.fname: filename for PRT.mat (string)

`in.model_name`: name for this model (string)

Outputs:

Writes the following fields in the PRT data structure:

```
PRT.model(m).output.fold(i).targets:    targets for fold(i)
PRT.model(m).output.fold(i).predictions: predictions for fold(i)
PRT.model(m).output.fold(i).stats:      statistics for fold(i)
PRT.model(m).output.fold(i).custom:     optional fields
```

Notes: - The `PRT.model(m).input` fields are set by `prt_init_model`, not by this function

14.6 prt_cv_opt_param.m

Function to pass optional parameters into the classifier. This is primarily used for complex data prediction methods that need to know something about the experimental design that is normally not accessible to generic prediction functions (e.g. task onsets or TR). Examples of this kind of classifier include multi-class classifier using kernel regression (MCKR) and the machine that implements nested cross-validation.

Inputs:

```
PRT: main data structure
ID:  id matrix for the current cross-validation fold
CV:  cross-validation structure (current fold only)
```

Outputs:

Provides the following fields for use by the classifier

```
param.cv_fold:  the vector from the CV matrix controlling this CV fold
.id_fold:       the id matrix for this fold
.group:         group data for subjects included in this fold (copied
                from PRT)
.onsets:        matrix of event onsets (n_events x n_conditions)
.durations:     matrix of event durations (n_events x n_conditions)
.TR:            TR for this group/subject/modality/event
.unit:          units for this design 0 = scans, 1 = seconds
.ids:           reduced id matrix (group,subject,modality)
```

N.B.: - The `onsets`, `durations`, `TR`, `Unit` and `ids` fields will only be populated for subjects that have a design specified that has the same number of events for all subjects included. This should probably be improved in the future.

14.7 prt_data_conditions.m

PRT_DATA_CONDITIONS M-file for `prt_data_conditions.fig`

PRT_DATA_CONDITIONS, by itself, creates a new PRT_DATA_CONDITIONS or raises the existing singleton*.

`H = PRT_DATA_CONDITIONS` returns the handle to a new `PRT_DATA_CONDITIONS` or the handle to the existing singleton*.

`PRT_DATA_CONDITIONS('CALLBACK',hObject,eventData,handles,...)` calls the local function named `CALLBACK` in `PRT_DATA_CONDITIONS.M` with the given input arguments.

`PRT_DATA_CONDITIONS('Property','Value',...)` creates a new `PRT_DATA_CONDITIONS` or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before `prt_data_conditions_OpeningFcn` gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to `prt_data_conditions_OpeningFcn` via `varargin`.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: `GUIDE`, `GUIDATA`, `GUIHANDLES`

14.8 prt_data_modality.m

`PRT_DATA_MODALITY` M-file for `prt_data_modality.fig`

`PRT_DATA_MODALITY`, by itself, creates a new `PRT_DATA_MODALITY` or raises the existing singleton*.

`H = PRT_DATA_MODALITY` returns the handle to a new `PRT_DATA_MODALITY` or the handle to the existing singleton*.

`PRT_DATA_MODALITY('CALLBACK',hObject,eventData,handles,...)` calls the local function named `CALLBACK` in `PRT_DATA_MODALITY.M` with the given input arguments.

`PRT_DATA_MODALITY('Property','Value',...)` creates a new `PRT_DATA_MODALITY` or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before `prt_data_modality_OpeningFcn` gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to `prt_data_modality_OpeningFcn` via `varargin`.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: `GUIDE`, `GUIDATA`, `GUIHANDLES`

14.9 prt_data_review.m

`PRT_DATA_REVIEW` M-file for `prt_data_review.fig`

`PRT_DATA_REVIEW`, by itself, creates a new `PRT_DATA_REVIEW` or raises the existing singleton*.

`H = PRT_DATA_REVIEW` returns the handle to a new `PRT_DATA_REVIEW` or the handle to the existing singleton*.

PRT_DATA_REVIEW('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_DATA_REVIEW.M with the given input arguments.

PRT_DATA_REVIEW('Property','Value',...) creates a new PRT_DATA_REVIEW or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_data_review_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_data_review_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.10 prt_defaults.m

Sets the defaults which are used by PRoNTo, Pattern Recognition in Neuroimaging Toolbox

FORMAT prt_defaults

This file can be customised to any the site/person own setup. Individual users can make copies which can be stored on their own matlab path. Make sure your 'prt_defaults' is the first one found in the path. See matlab documentation for details on setting path.

Care must be taken when modifying this file!

The structure and content of this file are largely inspired by SPM:
<http://www.fil.ion.ucl.ac.uk/spm>

14.11 prt_fs.m

Function to build file arrays containing the (linearly detrended) data and compute a linear (dot product) kernel from them

Inputs:

in.fname: filename for the PRT.mat (string)
 in.fs_name: name of fs and relative path filename for the kernel matrix

in.mod(m).mod_name: name of modality to include in this kernel (string)
 in.mod(m).detrend: detrend (scalar: 0 = none, 1 = linear)
 in.mod(m).param_dt: parameters for the kernel detrend (e.g. DCT bases)
 in.mod(m).mode: 'all_cond' or 'all_scans' (string)
 in.mod(m).mask: mask file used to create the kernel
 in.mod(m).normalise: 0 = none, 1 = normalise_kernel, 2 = scale modality
 in.mod(m).matnorm: filename for scaling matrix

Outputs:

Calls prt_init_fs to populate basic fields in PRT.fs(f)...

Writes PRT.mat
 Writes the kernel matrix to the path indicated by in.fs_name

14.12 prt_func2html.m

Script to generate the list of .m functions into html files
 which can be browsed around with your favourite browser.

Note that this script relies on the M2HTML package which is **NOT**
 distributed with PRoNTTo!

For more information, please read the M2HTML tutorial and FAQ at:
 \$<\$<http://www.artefact.tk/software/matlab/m2html/>>\$

14.13 prt_get_defaults.m

Get/set the defaults values associated with an identifier

FORMAT defaults = prt_get_defaults
 Return the global "defaults" variable defined in prt_defaults.m.

FORMAT defval = prt_get_defaults(defstr)
 Return the defaults value associated with identifier "defstr".
 Currently, this is a '.' subscript reference into the global
 "prt_def" variable defined in prt_defaults.m.

FORMAT prt_get_defaults(defstr, defval)
 Sets the defaults value associated with identifier "defstr". The new
 defaults value applies immediately to:
 * new modules in batch jobs
 * modules in batch jobs that have not been saved yet
 This value will not be saved for future sessions of PRoNTTo. To make
 persistent changes, edit prt_defaults.m.

The structure and content of this file are largely inspired by SPM &
 Matlabbatch.

<http://www.fil.ion.ucl.ac.uk/spm>
<http://sourceforge.net/projects/matlabbatch/>

14.14 prt_get_filename.m

out = prt_get_filename(ids)

14.15 prt_init_fs.m

function to initialise the kernel data structure

FORMAT: Two modes are possible:

```

fid = prt_init_fs(PRT, in)
[fid, PRT, tocomp] = prt_init_fs(PRT, in)

```

USAGE 1:

function will return the id of a feature set or an error if it doesn't exist in PRT.mat

Input:

in.fs_name: name for the feature set (string)

Output:

fid : is the identifier for the feature set in PRT.mat

USAGE 2:

function will create the feature set in PRT.mat and overwrite it if it already exists.

Input:

in.fs_name: name for the feature set (string)

in.fname: name of PRT.mat

in.mod(m).mod_name: name of the modality

in.mod(m).detrend: type of detrending

in.mod(m).mode: 'all_scans' or 'all_cond'

in.mod(m).mask: mask used to create the feature set

in.mod(m).param_dt: parameters used for detrending (if any)

in.mod(m).normalise: scale the input scans or not

in.mod(m).matnorm: mat file used to scale the input scans

Output:

fid : is the identifier for the model constructed in PRT.mat

Populates the following fields in PRT.mat (copied from above):

PRT.fs(f).fs_name

PRT.fs(f).fas

PRT.fs(f).k_file

Also computes the following fields:

PRT.fs(f).id_mat: Identifier matrix (useful later)

PRT.fs(f).id_col_names: Columns in the id matrix

Note: this function does not write PRT.mat. That should be done by the calling function

14.16 prt_init_model.m

function to initialise the model data structure

FORMAT: Two modes are possible:

```
mid = prt_init_model(PRT, in)
```

```
[mid, PRT] = prt_init_model(PRT, in)
```

USAGE 1:

 function will return the id of a model or an error if it doesn't
 exist in PRT.mat

Input:

in.model_name: name of the model (string)

Output:

mid : is the identifier for the model in PRT.mat

USAGE 2:

 function will create the model in PRT.mat and overwrite it if it
 already exists.

Input:

in.model_name: name of the model to be created (string)
 in.use_kernel: use kernel or basis functions for this model (boolean)
 in.machine: prediction machine to use for this model (struct)
 in.type: 'classification' or 'regression'

Output:

Populates the following fields in PRT.mat (copied from above):
 PRT.model(m).input.model_name
 PRT.model(m).input.type
 PRT.model(m).input.use_kernel
 PRT.model(m).input.machine

Note: this function does not write PRT.mat. That should be done by the
 calling function

14.17 prt_latex.m

Extract information from the toolbox m-files and output them as usable
 .tex files which can be directly included in the manual.

There are 2 types of m2tex operations:

1. converting the job configuration tree, i.e. *_cfg_* files defining the
 batching interface into a series of .tex files.
 NOTE: Only generate .tex files for each exec_branch of prt_batch.
2. converting the help header of the functions into .tex files.

These files are then included in a manually written prt_manual.tex file,
 which also includes chapter/sections written manually.

File derived from that of the SPM8 distribution.
<http://www.fil.ion.ucl.ac.uk/spm>

14.18 prt_load.m

Function to load the PRT.mat and check its integrity regarding the kernels and feature sets that it is supposed to contain. Updates the set feature name if needed.

input : name of the PRT.mat, path included

output : PRT structure updated

14.19 prt_load_blocks.m

Load one or more blocks of data.

This script is a effectively a wrapper function that for the routines that actually do the work (SPM nifti routines)

The syntax is either:

`img = prt_load_blocks(filenamees, block_size, block_range)` just to specify continuous blocks of data

or

`img = prt_load_blocks(filenamees, voxel_index)` to access non continuous blocks

14.20 prt_model.m

Function to configure and build the PRT.model data structure

Input:

`model.fs(f).fs_name:` feature set(s) this CV approach is defined for
`model.fs(f).fs_features:` feature selection mode ('all' or 'mask')
`model.fs(f).mask_file:` mask for this feature set (fs_features='mask')

`in.fname:` filename for PRT.mat
`in.model_name:` name for this cross-validation structure
`in.type:` 'classification' or 'regression'
`in.use_kernel:` does this model use kernels or features?
`in.operations:` operations to apply before prediction

`in.fs(f).fs_name:` feature set(s) this CV approach is defined for

`in.class(c).class_name`
`in.class(c).group(g).subj(s).num`
`in.class(c).group(g).subj(s).modality(m).mod_name`
 EITHER: `in.class(c).group(g).subj(s).modality(m).conds(c).cond_name`
 OR: `in.class(c).group(g).subj(s).modality(m).all_scans`
 OR: `in.class(c).group(g).subj(s).modality(m).all_cond`

`in.cv.type:` type of cross-validation ('loso', 'logso', 'custom')
`in.cv.mat_file:` file specifying CV matrix (if type='custom');

Output:

This function performs the following functions:

1. populates basic fields in PRT.model(m).input
2. computes PRT.model(m).input.targets based on in.class(c)...
3. computes PRT.model(m).input.samp_idx based on targets
4. computes PRT.model(m).input.cv_mat based on the labels and CV spec

14.21 prt_normalise_kernel.m

FORMAT K_normalised = prt_normalise_kernel(K)

This function normalises the kernel matrix such that each entry is divided by the product of the std deviations, i.e.

$K_{\text{new}}(x,y) = K(x,y) / \sqrt{\text{var}(x)*\text{var}(y)}$

14.22 prt_permutation.m

Function to compute permutation test

Inputs:

PRT: PRT structured including model

n_permu: number of permutations

modelid: model ID

Outputs:

for classification

permutation.c_acc: Permuted accuracy per class

permutation.b_acc: Permuted balanced accuracy

permutation.pvalue.b_acc: p-value for c_acc

permutation.pvalue.c_acc: p-value for b_acc

for regression

permutation.corr: Permuted correlation

permutation.mse: Permuted mean square error

permutation.corr: p-value for corr

permutation.mse: p-value for mse

14.23 prt_preproc.m

Function to preprocess the images, by loading each one of them (or the ones corresponding to the selected scans when a design was specified), applying the masks on them and, if asked, detrend along each voxel along the time series.

INPUT:

fname filename and path to PRT.mat

OUTPUT:

results are saved on disk.

14.24 prt_remove_confound.m

```
[Kr, R] = prt_remove_confound(K,C)
```

Function to remove confounds from kernel.

14.25 prt_stats.m

Function to compute predictions machine performance statistics

Inputs:

model.predictions: predictions derived from the predictive model

model.type: what type of prediction machine (e.g. 'classifier','regression')

t: true targets

flag: 'fold' for statistics in each fold

'model' for statistics in each model

Outputs:

Classification:

stats.con_mat: Confusion matrix (nClasses x nClasses matrix, pred x true)

stats.acc: Accuracy (scalar)

stats.b_acc: Balanced accuracy (nClasses x 1 vector)

stats.c_acc: Accuracy by class (nClasses x 1 vector)

stats.c_pv: Predictive value for each class (nClasses x 1 vector)

Regression:

stats.mse: Mean square error between test and prediction

stats.corr: Correlation between test and prediction

14.26 prt_struct2latex.m

Function that takes in a structure S and writes down the latex code describing the whole structure and substructures recursively.

The routine specifically generates the 'adv_PRTstruct.tex' file that is included, in the prt_manual.

Bits of the code and copied/inspired by spm_latex.m from the SPM8 distribution: <http://www.fil.ion.ucl.ac.uk/spm>

14.27 prt_text_input.m

PRT_TEXT_INPUT M-file for prt_text_input.fig

PRT_TEXT_INPUT, by itself, creates a new PRT_TEXT_INPUT or raises the

existing singleton*.

`H = PRT_TEXT_INPUT` returns the handle to a new `PRT_TEXT_INPUT` or the handle to the existing singleton*.

`PRT_TEXT_INPUT('CALLBACK',hObject,eventData,handles,...)` calls the local function named `CALLBACK` in `PRT_TEXT_INPUT.M` with the given input arguments.

`PRT_TEXT_INPUT('Property','Value',...)` creates a new `PRT_TEXT_INPUT` or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before `prt_text_input_OpeningFcn` gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to `prt_text_input_OpeningFcn` via `varargin`.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: `GUIDE`, `GUIDATA`, `GUIHANDLES`

14.28 `prt_ui_compute_weights.m`

`PRT_UI_COMPUTE_WEIGHTS` M-file for `prt_ui_compute_weights.fig`

`PRT_UI_COMPUTE_WEIGHTS`, by itself, creates a new `PRT_UI_COMPUTE_WEIGHTS` or raises the existing singleton*.

`H = PRT_UI_COMPUTE_WEIGHTS` returns the handle to a new `PRT_UI_COMPUTE_WEIGHTS` or the handle to the existing singleton*.

`PRT_UI_COMPUTE_WEIGHTS('CALLBACK',hObject,eventData,handles,...)` calls the local function named `CALLBACK` in `PRT_UI_COMPUTE_WEIGHTS.M` with the given input arguments.

`PRT_UI_COMPUTE_WEIGHTS('Property','Value',...)` creates a new `PRT_UI_COMPUTE_WEIGHTS` or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before `prt_ui_compute_weights_OpeningFcn` gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to `prt_ui_compute_weights_OpeningFcn` via `varargin`.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: `GUIDE`, `GUIDATA`, `GUIHANDLES`

14.29 `prt_ui_cv_model.m`

`PRT_UI_CV_MODEL` M-file for `prt_ui_cv_model.fig`

`PRT_UI_CV_MODEL`, by itself, creates a new `PRT_UI_CV_MODEL` or raises the existing singleton*.

`H = PRT_UI_CV_MODEL` returns the handle to a new `PRT_UI_CV_MODEL` or the handle to the existing singleton*.

PRT_UI_CV_MODEL('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_CV_MODEL.M with the given input arguments.

PRT_UI_CV_MODEL('Property','Value',...) creates a new PRT_UI_CV_MODEL or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_ui_cv_model_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_ui_cv_model_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.30 prt_ui_design.m

PRT_UI_DESIGN M-file for prt_ui_design.fig

PRT_UI_DESIGN, by itself, creates a new PRT_UI_DESIGN or raises the existing singleton*.

H = PRT_UI_DESIGN returns the handle to a new PRT_UI_DESIGN or the handle to the existing singleton*.

PRT_UI_DESIGN('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_DESIGN.M with the given input arguments.

PRT_UI_DESIGN('Property','Value',...) creates a new PRT_UI_DESIGN or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_ui_design_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_ui_design_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.31 prt_ui_kernel_construction.m

PRT_UI_KERNEL MATLAB code for prt_ui_kernel.fig

PRT_UI_KERNEL, by itself, creates a new PRT_UI_KERNEL or raises the existing singleton*.

H = PRT_UI_KERNEL returns the handle to a new PRT_UI_KERNEL or the handle to the existing singleton*.

PRT_UI_KERNEL('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_KERNEL.M with the given input arguments.

`PRT_UI_KERNEL('Property','Value',...)` creates a new `PRT_UI_KERNEL` or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before `prt_ui_kernel_OpeningFcn` gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to `prt_ui_kernel_OpeningFcn` via `varargin`.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: `GUIDE`, `GUIDATA`, `GUIHANDLES`

14.32 `prt_ui_main.m`

`PRT_UI_MAIN` M-file for `prt_ui_main.fig`

`PRT_UI_MAIN`, by itself, creates a new `PRT_UI_MAIN` or raises the existing singleton*.

`H = PRT_UI_MAIN` returns the handle to a new `PRT_UI_MAIN` or the handle to the existing singleton*.

`PRT_UI_MAIN('CALLBACK',hObject,eventData,handles,...)` calls the local function named `CALLBACK` in `PRT_UI_MAIN.M` with the given input arguments.

`PRT_UI_MAIN('Property','Value',...)` creates a new `PRT_UI_MAIN` or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before `prt_ui_main_OpeningFcn` gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to `prt_ui_main_OpeningFcn` via `varargin`.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: `GUIDE`, `GUIDATA`, `GUIHANDLES`

14.33 `prt_ui_model.m`

`PRT_UI_KERNEL_CONSTRUCTION` M-file for `prt_ui_kernel.construction.fig`

`PRT_UI_KERNEL_CONSTRUCTION`, by itself, creates a new `PRT_UI_KERNEL_CONSTRUCTION` or raises the existing singleton*.

`H = PRT_UI_KERNEL_CONSTRUCTION` returns the handle to a new `PRT_UI_KERNEL_CONSTRUCTION` or the handle to the existing singleton*.

`PRT_UI_KERNEL_CONSTRUCTION('CALLBACK',hObject,eventData,handles,...)` calls the local function named `CALLBACK` in `PRT_UI_KERNEL_CONSTRUCTION.M` with the given input arguments.

`PRT_UI_KERNEL_CONSTRUCTION('Property','Value',...)` creates a new `PRT_UI_KERNEL_CONSTRUCTION` or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before `prt_ui_kernel_construction_OpeningFcn` gets called. An unrecognized property name or invalid value makes property application stop. All

inputs are passed to `prt_ui_kernel_construction_OpeningFcn` via `varargin`.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.34 `prt_ui_prepare_data.m`

PRT_UI_KERNEL MATLAB code for `prt_ui_kernel.fig`

PRT_UI_KERNEL, by itself, creates a new PRT_UI_KERNEL or raises the existing singleton*.

H = PRT_UI_KERNEL returns the handle to a new PRT_UI_KERNEL or the handle to the existing singleton*.

PRT_UI_KERNEL('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_KERNEL.M with the given input arguments.

PRT_UI_KERNEL('Property','Value',...) creates a new PRT_UI_KERNEL or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before `prt_ui_kernel_OpeningFcn` gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to `prt_ui_kernel_OpeningFcn` via `varargin`.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.35 `prt_ui_prepare_datamod.m`

PRT_UI_KERNEL_MODALITY M-file for `prt_ui_kernel_modality.fig`

PRT_UI_KERNEL_MODALITY, by itself, creates a new PRT_UI_KERNEL_MODALITY or raises the existing singleton*.

H = PRT_UI_KERNEL_MODALITY returns the handle to a new PRT_UI_KERNEL_MODALITY or the handle to the existing singleton*.

PRT_UI_KERNEL_MODALITY('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_KERNEL_MODALITY.M with the given input arguments.

PRT_UI_KERNEL_MODALITY('Property','Value',...) creates a new PRT_UI_KERNEL_MODALITY or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before `prt_ui_kernel_modality_OpeningFcn` gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to `prt_ui_kernel_modality_OpeningFcn` via `varargin`.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.36 prt_ui_results.m

PRT_UI_RESULTS MATLAB code for prt_ui_results.fig

PRT_UI_RESULTS, by itself, creates a new PRT_UI_RESULTS or raises the existing singleton*.

H = PRT_UI_RESULTS returns the handle to a new PRT_UI_RESULTS or the handle to the existing singleton*.

PRT_UI_RESULTS('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_RESULTS.M with the given input arguments.

PRT_UI_RESULTS('Property','Value',...) creates a new PRT_UI_RESULTS or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_ui_results_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_ui_results_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.37 prt_ui_reviewCV.m

PRT_UI_REVIEWCV M-file for prt_ui_reviewCV.fig

PRT_UI_REVIEWCV, by itself, creates a new PRT_UI_REVIEWCV or raises the existing singleton*.

H = PRT_UI_REVIEWCV returns the handle to a new PRT_UI_REVIEWCV or the handle to the existing singleton*.

PRT_UI_REVIEWCV('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_REVIEWCV.M with the given input arguments.

PRT_UI_REVIEWCV('Property','Value',...) creates a new PRT_UI_REVIEWCV or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_ui_reviewCV_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_ui_reviewCV_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.38 prt_ui_reviewmodel.m

PRT_UI_REVIEWMODEL M-file for prt_ui_reviewmodel.fig

PRT_UI_REVIEWMODEL, by itself, creates a new PRT_UI_REVIEWMODEL or raises the existing singleton*.

H = PRT_UI_REVIEWMODEL returns the handle to a new PRT_UI_REVIEWMODEL or the handle to the existing singleton*.

PRT_UI_REVIEWMODEL('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_REVIEWMODEL.M with the given input arguments.

PRT_UI_REVIEWMODEL('Property','Value',...) creates a new PRT_UI_REVIEWMODEL or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_ui_reviewmodel_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_ui_reviewmodel_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.39 prt_ui_select_class.m

PRT_UI_SELECT_CLASS M-file for prt_ui_select_class.fig

PRT_UI_SELECT_CLASS, by itself, creates a new PRT_UI_SELECT_CLASS or raises the existing singleton*.

H = PRT_UI_SELECT_CLASS returns the handle to a new PRT_UI_SELECT_CLASS or the handle to the existing singleton*.

PRT_UI_SELECT_CLASS('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_SELECT_CLASS.M with the given input arguments.

PRT_UI_SELECT_CLASS('Property','Value',...) creates a new PRT_UI_SELECT_CLASS or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_ui_select_class_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_ui_select_class_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.40 prt_ui_select_reg.m

PRT_UI_SELECT_REG M-file for prt_ui_select_reg.fig

PRT_UI_SELECT_REG, by itself, creates a new PRT_UI_SELECT_REG or raises the existing singleton*.

H = PRT_UI_SELECT_REG returns the handle to a new PRT_UI_SELECT_REG or the handle to the existing singleton*.

PRT_UI_SELECT_REG('CALLBACK', hObject, eventData, handles,...) calls the local function named CALLBACK in PRT_UI_SELECT_REG.M with the given input arguments.

PRT_UI_SELECT_REG('Property','Value',...) creates a new PRT_UI_SELECT_REG or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_ui_select_reg_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_ui_select_reg_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.41 machines

14.41.1 machines\prt_KRR.m

w = prt_KRR(K,t,reg)

14.41.2 machines\prt_machine.m

Run machine function for classification or regression

FORMAT output = prt_machine(d,m)

Inputs:

d - structure with information about the data, with fields:
 Mandatory fields:
 .train - training data (cell array of matrices of row vectors, each [Ntr x D]). each matrix contains one representation of the data. This is useful for approaches such as multiple kernel learning.
 .test - testing data (cell array of matrices row vectors, each [Nte x D])
 .tr_targets - training labels (for classification) or values (for regression) (column vector, [Ntr x 1])
 .use_kernel - flag, is data in form of kernel matrices (true) or in form of features (false)
 Optional fields: the machine is responsible for dealing with this optional fields (e.g. d.testcov)
 m - structure with information about the classification or regression machine to use, with fields:
 .function - function for classification or regression (string)

.args - function arguments (either a string, a matrix, or a struct). This is specific to each machine, e.g. for an L2-norm linear SVM this could be the C parameter

Output:

output - output of machine (struct).

Mandatory fields:

.predictions - predictions of classification or regression
[Nte x D]

Optional fields: the machine is responsible for returning parameters of interest. For exemple for an SVM this could be the number of support vector used in the hyperplane weights computation

14.41.3 machines\prt_machine_RT_bin.m

Run binary Ensemble of Regression Tree - wrapper for Pierre Geurt's RT code

FORMAT output = prt_machine_RT_bin(d,args)

Inputs:

d - structure with data information, with mandatory fields:

- .train - training data (cell array of matrices of row vectors, each [Ntr x D]). each matrix contains one representation of the data. This is useful for approaches such as multiple kernel learning.
- .test - testing data (cell array of matrices row vectors, each [Nte x D])
- .tr_targets - training labels (for classification) or values (for regression) (column vector, [Ntr x 1])
- .use_kernel - flag, is data in form of kernel matrices (true) or in form of features (false)

args - vector of RT arguments

- args(1) - number of trees (default: 501)

Output:

output - output of machine (struct).

* Mandatory fields:

.predictions - predictions of classification or regression [Nte x D]

* Optional fields:

.func_val - value of the decision function

.type - which type of machine this is (here, 'classifier')

14.41.4 machines\prt_machine_gpml.m

Run binary SVM - wrapper for libSVM

FORMAT output = prt_machine_gpml(d,args)

Inputs:

d - structure with data information, with mandatory fields:

- .train - training data (cell array of matrices of row vectors, each [Ntr x D]). each matrix contains one representation of the data. This is useful for approaches such as multiple kernel learning.
- .test - testing data (cell array of matrices row vectors, each [Nte x D])
- .testcov - testing covariance (cell array of matrices row vectors, each [Nte x Nte])
- .tr_targets - training labels (for classification) or values (for regression) (column vector, [Ntr x 1])

```

    .use_kernel - flag, is data in form of kernel matrices (true) of in
                  form of features (false)
args      - argument string, where
    -h      - optimise hyperparameters (otherwise don't)
    -l lik   - likelihood function. Currently only lik = 'erf' is
                  supported
    -c cov   - covariance function:
                  'lin' - simple dot product (no hyperparameters)
                  'linb' - dot product with bias (one hyperparameter)

```

Output:

```

output - output of machine (struct).
* Mandatory fields:
    .predictions - predictions of classification or regression [Nte x D]
* Optional fields:
    .type        - which type of machine this is (here, 'classifier')
    .p           - predictive probabilities
    .loghyper    - log hyperparameters
    .nlml        - negative log marginal likelihood
    .alpha       - GP weighting coefficients
    .sW          - likelihood matrix (see Rasmussen & Williams, 2006)
    .L           - Cholesky factor

```

14.41.5 machines\prt_machine_krr.m

Kernel ridge regression

FORMAT output = prt_machine_svm_bin(d,args)

Inputs:

```

d      - structure with data information, with mandatory fields:
    .train      - training data (cell array of matrices of row vectors,
                  each [Ntr x D]). each matrix contains one representation
                  of the data. This is useful for approaches such as
                  multiple kernel learning.
    .test       - testing data (cell array of matrices row vectors, each
                  [Nte x D])
    .tr_targets - training labels (for classification) or values (for
                  regression) (column vector, [Ntr x 1])
    .use_kernel - flag, is data in form of kernel matrices (true) of in
                  form of features (false)
args    - libSVM arguments

```

Output:

```

output - output of machine (struct).
* Mandatory fields:
    .predictions - predictions of classification or regression [Nte x D]
* Optional fields:
    .func_val - value of the decision function
    .type     - which type of machine this is (here, 'classifier')

```

14.41.6 machines\prt_machine_rvr.m

Relevance vector regression (training and testing)

FORMAT output = prt_machine_svm_bin(d,args)

Inputs:

```

d      - structure with data information, with mandatory fields:
    .train      - training data (cell array of matrices of row vectors,

```

```

        each [Ntr x D]). each matrix contains one representation
        of the data. This is useful for approaches such as
        multiple kernel learning.
    .test      - testing data (cell array of matrices row vectors, each
                [Nte x D])
    .tr_targets - training labels (for classification) or values (for
                regression) (column vector, [Ntr x 1])
    .use_kernel - flag, is data in form of kernel matrices (true) or in
                form of features (false)
    args      - libSVM arguments
Output:
    output - output of machine (struct).
    * Mandatory fields:
    .predictions - predictions of classification or regression [Nte x D]
    * Optional fields:
    .func_val - value of the decision function
    .type      - which type of machine this is (here, 'classifier')

```

14.41.7 machines\prt_machine_svm_bin.m

```

Run binary SVM - wrapper for libSVM
FORMAT output = prt_machine_svm_bin(d,args)
Inputs:
    d      - structure with data information, with mandatory fields:
    .train  - training data (cell array of matrices of row vectors,
                each [Ntr x D]). each matrix contains one representation
                of the data. This is useful for approaches such as
                multiple kernel learning.
    .test   - testing data (cell array of matrices row vectors, each
                [Nte x D])
    .tr_targets - training labels (for classification) or values (for
                regression) (column vector, [Ntr x 1])
    .use_kernel - flag, is data in form of kernel matrices (true) or in
                form of features (false)
    args    - libSVM arguments
Output:
    output - output of machine (struct).
    * Mandatory fields:
    .predictions - predictions of classification or regression [Nte x D]
    * Optional fields:
    .func_val - value of the decision function
    .type      - which type of machine this is (here, 'classifier')

```

14.41.8 machines\prt_rvr.m

Optimisation for Relevance Vector Regression

```

[w,alpha,beta,ll]=spm_rvr(Phi,t)
Phi  - MxM matrix derived from kernel function of vector pairs
t    - the values to be matched
w    - weights
alpha - 1/variance for the prior part of the model
beta  - 1/variance for the likelihood part of the model
ll    - the negative log-likelihood.

```

```
[w,alpha,beta,nu,ll]=spm_rvr(K,t,opt)
K      - a cell-array of MxM dot-product matrices.
t      - the values to be matched
opt    - either 'Linear' or 'Gaussian RBF'
          'Linear'      is for linear regression models, where
                        the optimal kernel is generated by
                        [nu(1)*K1 + nu(1)*K2... ones(size(K1,1),1)]
          'Gaussian RBF' is for regression using Gaussian radial basis
                        functions. The kernel is generated from
                        P1 = nu(1)*K1 + nu(1)*K2 ... ;
                        P2 = repmat(diag(P1) ,1,size(P1,2)) +...
                        repmat(diag(P1)',size(P1,1),1) - 2*P1;
                        Phi = exp([-0.5*P2 ones(size(P1,1),1)]);
w      - weights
alpha  - 1/variance for the prior part of the model
beta   - 1/variance for the likelihood part of the model
nu     - parameters that convert the dot-product matrices into
        a kernel matrix (Phi).
ll     - the negative log-likelihood.
```

The first way of calling the routine simply optimises the weights. This involves estimating a restricted maximum likelihood (REML) solution, which maximises $P(\alpha, \beta | t, \Phi)$. Note that REML is also known as Type II Maximum Likelihood (ML-II). The ML-II solution tends towards infinite weights for some the regularisation terms (i.e. $1/\alpha(i)$ approaches 0). The appropriate columns are removed from the model when this happens.

The second way of calling the routine also estimates additional input scale parameters as described in Appendix C of Tipping (2001). This method is much slower, as a full optimisation for the scale parameters is done after each update of the alphas and beta.

see: <http://research.microsoft.com/mlp/RVM/relevance.htm>

Refs:

The Relevance Vector Machine.

In S. A. Solla, T. K. Leen, and K.-R. Mller (Eds.),
Advances in Neural Information Processing Systems 12,
pp. 652-658. Cambridge, Mass: MIT Press.

Michael E. Tipping

Sparse Bayesian Learning and the Relevance Vector Machine
Journal of Machine Learning Research 1 (2001) 211-244

14.41.9 machines\prt_weights.m

Run function to compute weights

FORMAT weights = prt_weights(d,m)

Inputs:

```
d      - data structure
m      - machine structure
        .function - function to compute weights (string)
        .args     - function arguments
```

Output:

weights - weights vector [Nfeatures x 1]

14.41.10 machines\prt_weights_bin_linkernel.m

Run function to compute weights for linear kernel binary classifiers

FORMAT weights = prt_weights_bin_linkernel (d,args)

Inputs:

d - data structure
 .datamat - data matrix [Nfeatures x Nexamples]
 .coeffs - coefficients vector [Nexamples x 1]
 args - function arguments (can be empty)

Output:

weights - vector with weights [Nfeatures x 1]

14.41.11 machines\prt_weights_svm_bin.m

Run function to compute weights for binary SVM

FORMAT weights = prt_weights_svm_bin (d,args)

Inputs:

d - data structure
 .datamat - data matrix [Nfeatures x Nexamples]
 .coeffs - coefficients vector [Nexamples x 1]
 args - function arguments (can be empty)

Output:

weights - vector with weights [Nfeatures x 1]

14.42 utils

14.42.1 utils\prt_centre_kernel.m

This function centres the kernel matrix, respecting the independence of training and test partitions. See Shawe-Taylor and Cristianini for background on this approach.

Shawe-Taylor, J. and Cristianini, N. (2004). Kernel methods for Pattern analysis. Cambridge University Press.

14.42.2 utils\prt_checkAlphaNumUnder.m

check whether a given string is alphanumerical or underscore

FORMAT out = prt_checkAlphaNumUnder(s)

Inputs:

s - a string of arbitrary length to check

Output:

out - logical 1 if the all chars in the string are alphanumerical
 logical 0 otherwise

Based on isalpha_num in the identification toolbox

14.42.3 utils\prt_normalise_kernel.m

This function normalises the kernel matrix such that each entry is

divided by the product of the std deviations, i.e.
 $K_{\text{new}}(x,y) = K(x,y) / \text{sqrt}(\text{var}(x)*\text{var}(y))$

Part V

Bibliography

Bibliography

- [1] Christopher M. Bishop. *Pattern Recognition and Machine learning*. Springer, 2006.
- [2] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, April 2006.
- [3] Janaina Mourao-Miranda, Karl J Friston, and Michael Brammer. Dynamic discrimination analysis: a spatial-temporal svm. *Neuroimage*, 36(1):88–99, May 2007.
- [4] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. Adaptive Computation and Machine Learning. the MIT Press,, 2006.